

NHAI Hackathon 7.0

Offline Face Recognition + Liveness Detection
Strategy, Research & Architecture

Participant: Sahil Chandel (Solo)

Submission Window: 2026-05-22 → 2026-06-05

Target App: NHAI Datalake 3.0 (React Native)

Document Date: May 2026

NHAI Hackathon 7.0 — Strategy, Research & Architecture

Problem: Mobile-based secure offline facial recognition + liveness detection system for remote locations (NHAI Datalake 3.0 integration). **Participant:** Sahil Chandel (Solo) **Submission window:** 2026-05-22 → 2026-06-05
Document compiled: May 2026

Table of Contents

1. [Executive Summary](#)
 2. [Hackathon Brief Recap](#)
 3. [Win Probability Assessment](#)
 4. [Global Research — Academic Papers](#)
 5. [Global Research — Open-Source GitHub Repos](#)
 6. [Global Research — Country-Wise Deployments](#)
 7. [Comparative Landscape & The Exploitable Gap](#)
 8. [Winning Technology Stack](#)
 9. [System Architecture](#)
 10. [Data Flow — Verification Sequence](#)
 11. [Project File Structure](#)
 12. [Module Specifications](#)
 13. [Cross-Cutting Concerns](#)
 14. [Performance Budget](#)
 15. [NHAI Evaluation Criteria Mapping](#)
 16. [14-Day Build Roadmap](#)
 17. [Pitch Slides Outline](#)
 18. [Honest Gaps to Defend in Q&A](#)
 19. [Consolidated Sources](#)
-

1. Executive Summary

Win probability: 70-75% (solo).

The hackathon problem is solvable with assembly of existing open-source components — no fundamental R&D required. The committed stack uses **EdgeFace-XS (IJCBB-2023 winner) + MiniFASNetV2 + V1SE + MediaPipe FaceMesh challenges**, totaling **~2.6 MB INT8 model footprint** (87% under the 20 MB limit). End-to-end inference target: **150-300 ms** on a mid-range Android device (Snapdragon 6xx / Helio G70 class).

The exploitable gap: No production OSS system worldwide combines (a) offline 1:N face match on phone, (b) Indian-demographic-tuned model, (c) React Native + AWS sync loop — all in <20 MB and <1 s. DigiYatra is

closest (1:1, proprietary). NMMS/POSHAN/AEBAS fail at exactly the zero-network field scenarios NHAI cares about.

Pitch angle: *“DigiYatra’s offline cousin for field workers”* — politically resonant for NHAI evaluators.

2. Hackathon Brief Recap

Requirement	Specification
Framework	React Native (Android 8.0+, iOS 12+)
Model size	~20 MB target (smaller is better)
Inference latency	< 1 second end-to-end
Accuracy	> 95% on diverse Indian demographics, robust to outdoor lighting (harsh sun, low light, shadows)
Hardware	3 GB RAM minimum, no high-end GPU dependency
Licensing	100% open-source only, no commercial licenses
Liveness	Offline anti-spoofing (blink/smile/head-turn challenges)
Sync	AWS sync-and-purge after connectivity restored
Deliverables	RN prototype + source code + PPT/PDF + technical documentation

Evaluation criteria (100 marks total): - Innovation (30) — edge AI model efficiency, compression, liveness effectiveness - Feasibility (30) — RN integration ease, <1s on mid-range devices - Scalability & Sustainability (20) — offline-to-online sync reliability, demographic/lighting adaptability - Presentation (20) — code clarity, integration guide, final pitch

3. Win Probability Assessment

Factor	Verdict
Open-source stack maturity	✔ Strong — full pipeline buildable from MIT/Apache repos
Model size budget	✔ Massive headroom — 2.6 MB vs 20 MB target
Latency budget	✔ Achievable — ~300 ms total
Indian demographic data	✔ IndicFairFace + JFAD + DFW available
Sahil’s RN experience	✔ Confirmed solid
Sahil’s prior NHA1 work	✔ YOLOv8 highway inspection — domain credibility
Solo execution risk	⚠ Tight 14-day window, no buffer for major blockers
MiniFASNet outdoor harsh-sun perf	⚠ Not validated in published literature — must field-test
Anti-spoofing under adversarial attacks	⚠ RGB-only, no depth — limited threat model

4. Global Research — Academic Papers

4.1 Lightweight On-Device Face Recognition Models

Model	Params	Accuracy	Latency	Paper
EdgeFace-S ($\gamma=0.5$)	1.77M	99.73% LFW, 94.85% IJB-C	—	arXiv 2307.01838 — IDIAP, IJCB-2023 winner
MobileFaceNets	1.0M	99.55% LFW	18 ms on phone	arXiv 1804.07573
FaceLiVT-S	—	99.6% LFW, 93.9% CFP-FP	0.47 ms iPhone 15 Pro	arXiv 2506.10361 — IEEE ICIP 2025
GhostFaceNets++	—	SOTA at <4 MB	51-62 MFLOPs	IEEE Access 2023/2024
MixFaceNets	~1M	beats ProxylessFaceNAS at <500 MFLOPs	—	arXiv 2107.13046 — IJCB 2021
PocketNetS-128	0.92M	99.58% LFW	587 MFLOPs	arXiv 2108.10710 — IEEE TBIOM 2022
MobiFace	—	99.73% LFW, 91.3% MegaFace	~7.8 MB	arXiv 1811.11080
QuantFace	low-bit (down to 6-bit)	preserves accuracy	uses synthetic data only	arXiv 2206.10526 — IJCB 2022

4.2 Face Anti-Spoofing / Presentation Attack Detection (PAD) for Mobile

Method	Size/Specs	Performance	Paper / Repo
MiniFASNetV1 / V2	0.41M / 0.43M params (~1.5 MB ea)	97.8% TPR @ FPR 1e-5, 20 ms	Silent-Face-Anti-Spoofing — Apache-2.0
CDCN / CDCN++	heavier	SOTA OULU-NPU/SiW; 1st place ChaLearn 2020	arXiv 2003.04092 — CVPR 2020
FAS-SGTD	multi-frame	SOTA on OULU-NPU/SiW/CASIA/Replay	arXiv 2003.08061 — CVPR 2020
DeepPixBiS	~1.4M	strong inductive bias for mobile	ICB 2019
Liveness with Randomized Challenge-Response	active, no model	99% accuracy on photo/video attacks	IJICIC 2023
EAR-based blink	lightweight (landmark-based)	~90% on EyeBlink8	Soukupova & Cech 2016
Aurora Guard	screen-illumination cue	—	arXiv 2102.00713

4.3 Model Compression for Edge Face Recognition

Technique	Result	Source
ArcFace INT8 (ONNX Model Zoo)	4× smaller, 1.78× speedup, 0.02% embedding error	HuggingFace
TFLite/LiteRT 8-bit spec	INT8 reference on ARM with XNNPACK	Google AI Edge docs
KD for face model compression	student-teacher (PocketNet/MixFaceNet families)	arXiv 1906.00619
Bridge Distillation	0.21M params, 0.057 MB, 763 fps on phone	arXiv 2409.11786

4.4 Indian Demographic Face Recognition

Resource	Size	Notes
JFAD (Jodhpur Faces of Academia)	40 subjects × 10 imgs	IIT Jodhpur, Dec 2024 — direct evidence LFW underrepresents Indian features. arXiv 2412.08048
IndicFairFace	14,400 images, 28 states + 8 UTs	arXiv 2602.12659
IMFDB (Indian Movie Face Database)	34k images, 100 actors	IIIT-H/CVIT — large pose/illumination/occlusion
DFW (Disguised Faces in the Wild)	11,157 imgs / 1,000 IDs	IIIT-Delhi — helmets, sunglasses, masks. arXiv 1811.08837 — critical for highway field robustness
NIST FRVT Demographics (NIST IR 8280, 8429)	—	Elevated FMR for South Asian and Indigenous groups documented

4.5 Best-Stack Recommendation (from papers research)

- **Detection:** BlazeFace or YuNet (~1.5 MB TFLite)
- **Recognition (primary):** EdgeFace-S($\gamma=0.5$) INT8 — ~2 MB, MIT license
- **Recognition (alt 2025):** FaceLiVT-S if code releases
- **Recognition (fallback):** MobileFaceNet INT8 with ArcFace head
- **Anti-spoofing passive:** MiniFASNetV2 + V1SE ensemble (Apache-2.0)
- **Anti-spoofing active:** MediaPipe FaceMesh + random {blink, smile, head-turn} challenges

5. Global Research — Open-Source GitHub Repos

5.1 React Native Camera + ML Runtime

Repo	Stars	License	Purpose
mrousavy/react-native-vision-camera	9.4k	MIT	High-perf RN camera with JS-worklet frame processors
mrousavy/react-native-fast-tflite	1.2k	MIT	Nitro TFLite runtime, zero-copy ArrayBuffers, CoreML/Metal/OpenGL delegates
luicfrr/react-native-vision-camera-face-detector	311	MIT	MLKit face detector frame-processor plugin
pedrol2b/react-native-vision-camera-mlkit	49	MIT	Broader MLKit features

5.2 Lightweight Face Detection Models

Repo / Model	Stars	License	Size	Notes
Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB	7.5k	MIT	1.04 MB FP32 / ~300 KB INT8	ONNX/NCNN/MNN/TFLite/Caffe
opencv/opencv_zoo — YuNet	1.3k	Apache-2.0	~340 KB ONNX, ~100 KB INT8	YuNet AP 0.834 on WIDER, better than ULFG
google-ai-edge/mediapipe — BlazeFace	35.3k	Apache-2.0	~225 KB TFLite	Best baseline for selfie distance
biubug6/Pytorch_Retinaface	2.7k	MIT	1.7 MB	WIDER hard 80.99%
deepinsight/insightface — SCRFD-500MF	28.8k	MIT (code)	~2.4 MB	Beats RetinaFace-Mobile at similar FLOPs

5.3 Lightweight Face Recognition Models

Repo / Model	Stars	License	Size	Notes
otroshi/edgeface	150	MIT	1.24M / 1.77M / 3.65M params	Direct fit — IJCB-2023 winner
HamadYA/GhostFaceNet	276	MIT	4-10 MB	LFW 99.78%, IJB-C 94.94%
deepinsight/insightface — buffalo_s/sc	28.8k	MIT code, but weights require Nov 2025 licensing email	—	⚠️ Avoid for commercial NHA1 deployment
timesler/facenet-pytorch	5.1k	MIT	107 MB	Too big for mobile
ageitgey/face_recognition	53k	MIT/PD	22 MB dlib	Avoid for mobile, no TFLite

5.4 Anti-Spoofing / Liveness

Repo	Stars	License	Notes
minivision-ai/Silent-Face-Anti-Spoofing	1.7k	Apache-2.0	MiniFASNet V1/V2 — production-grade Android APK reference
minivision-ai/Silent-Face-Anti-Spoofing-APK	—	Apache-2.0	NCNN-based reference; mine for integration patterns
shubham0204/OnDevice-Face-Recognition-Android	163	Apache-2.0	Best end-to-end reference — FaceNet TFLite + ObjectBox + MiniFASNet
syaringan357/Android-MobileFaceNet-MTCNN-FaceAntiSpoofing	279	MIT	Full Android prior art (older)
ZitongYu/CDCN	608	research	Reference only, no mobile path

5.5 Conversion & Quantization Tooling

Repo	License	Purpose
PINTO0309/onnx2tf	MIT	ONNX → TFLite/Keras with INT8 calibration
NXP/eiq-onnx2tflite	BSD-3	Alternative INT8 face-model conversion
Apple coremltools	BSD-3	iOS CoreML conversion path

5.6 Recommended GitHub Stack (7 repos)

1. [mrousavy/react-native-vision-camera](#) — camera + frame-processor harness
2. [mrousavy/react-native-fast-tflite](#) — on-device TFLite inference
3. [luicfrr/react-native-vision-camera-face-detector](#) — fast MLKit face detection
4. [opencv/opencv_zoo](#) (YuNet) OR [Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB](#) — fallback TFLite detector
5. [otroshi/edgeface](#) — primary recognition model
6. [minivision-ai/Silent-Face-Anti-Spoofing](#) — passive liveness
7. [shubham0204/OnDevice-Face-Recognition-Android](#) — end-to-end architecture mirror
8. [PINTO0309/onnx2tf](#) — build-time quantization

Total weights footprint: YuNet INT8 (~0.1 MB) + EdgeFace-XS INT8 (~2 MB) + MiniFASNetV2 INT8 (~0.5 MB) = ~2.6 MB of model weights.

6. Global Research — Country-Wise Deployments

6.1 India

System	Architecture	Useful for NHAI?
UIDAI Aadhaar Face Auth (Face RD app)	Liveness on-device, 1:1 match at UIDAI CIDR	Hybrid — not match-offline
AEBAS face mode	On-device liveness, match at UIDAI	Hybrid — not match-offline
NHAI Datalake 3.0	Already deployed; face attendance + Aadhaar + GPS for field staff	Position as offline-first companion to existing app
DigiYatra	Template in phone enclave, pass/fail to gate, 24h purge	Architectural template to copy — decentralized + politically safe
NMMS (MGNREGA), POSHAN Tracker	Geotagged photo + connectivity required	Pain point validation — widely criticized for zero-network failures
HyperVerge (commercial)	On-device liveness + biometry, 99.5% claimed	Won IndiaAI Face Auth Challenge — domestic benchmark
IDfy, Signzy	Primarily cloud APIs	Limited on-device match
FaceX.ai, Innefu Face2.0, Staqu Jarvis	Enterprise / govt RFP-driven	Thin public surface

6.2 USA

System	Architecture
Apple Face ID / Secure Enclave	TrueDepth + IR + Neural Engine match; never leaves device — gold standard
Google ML Kit Face Detection	Fully on-device, free; detection only , not recognition
AWS Rekognition	Pure cloud; useful only as sync target
NIST FRVT	NEC (Japan) #1 with 0.07% error across 12M faces (May 2026)
Open mobile ML stack	react-native-fast-tflite , mobilesec/arcface-tensorflowlite , Esteban Uri's Medium walkthroughs

6.3 Canada

System	Architecture
SecureKey (Interac, 2023)	Verified.Me + Onfido face + document, cloud-bound
CBSA Primary Inspection Kiosks	Live face → chip-embedded passport photo (kiosk) — document-tethered liveness pattern reusable for Aadhaar QR
Vector Institute / UToronto / Waterloo	Adversarial-robustness research, no flagship mobile FAS product

6.4 China

System	Architecture
Megvii (Face++)	“Embedded SDK Empowerment” — offline SDKs to OEMs; anti-spoof cloud/edge/embedded
SenseTime	Full-stack mobile SDK incl. masked-face FR
CloudWalk, Yitu	Mostly smart-city; mobile SDKs under license
SeetaFace (CAS)	BSD-2 GitHub — pure C++, zero deps, CPU-only, 97.1% LFW, 120 ms on i7 — best truly-open Chinese option
Smartphone OEM face unlock (Huawei/Xiaomi/OPPO/Vivo)	Fully on-device, 2D + IR
Alipay/WeChat face pay	Kiosk 3D depth cameras + server match — not useful offline pattern

6.5 UAE

System	Architecture
UAE Pass	Face capture matched to Emirates ID photo, cloud-anchored
Smart Salem	Kiosk biometric for Emirates ID
Emirates / Etihad biometric boarding	Airport-side cameras (Smart Tunnel, One ID)
MBZUAI	Top-15 CV research; no flagship public mobile FAS SDK

6.6 Japan

System	Architecture
NEC NeoFace	NIST FRVT #1; NeoFace Smart ID mobile app with limited offline cache
Panasonic FacePRO	Domestic surveillance
JAL / ANA Face Express	NEC-built; kiosk registration, gate verification, 24h purge
MyNumber + Apple Wallet	Face ID/Touch ID gates digital MyNumber credential (world-first outside US, 2025)

6.7 Russia

System	Architecture
VisionLabs LUNA SDK / LUNA ID	Android + iOS, OneShotLiveness offline on-device — closest functional twin to NHAI requirement
NtechLab FindFace SDK	C library, feature-vector based; powers Moscow Metro FacePay
Moscow Metro FacePay	Server-side match in VTB Bank Transport Processing

6.8 Community Blogs (Technically Meaty)

- Marc Rousavy (mrousavy.com) — author of `react-native-fast-tflite` and `react-native-vision-camera`; single most important RN-mobile-ML author
 - Esteban Uri (Medium) — [Real time face recognition with Android + TensorFlow Lite](#)
 - Ashraz Rashid (Medium, India) — [Building a React Native Face Detection Component from Scratch](#)
 - `thelamina` (Dev.to) — [How to Implement Face Detection in React Native Using React Native Vision Camera](#)
 - `yoshan0921` (Dev.to) — [Implement Face Detection App on iOS with React Native + Expo within 10 Minutes](#)
-

7. Comparative Landscape & The Exploitable Gap

Country	Flagship Pattern	Open / Licensable?	What India Team Can Learn
India (DigiYatra)	Template in phone enclave; pass/fail to gate; 24h purge	Closed	Decentralized template + signed result is politically safe architecture
USA (Apple/Google)	Secure Enclave + Neural Engine match	ML Kit free; Apple closed	Full RN SDK stack (Rousavy) is ours to use
Canada (CBSA, Interac)	Document-chip-as-trust-anchor	Closed	Tether liveness to signed credential (Aadhaar QR $\approx$ passport chip)
China (Megvii, Seeta)	Embedded SDK on OEM silicon	SeetaFace BSD-2; Megvii license	SeetaFace as proven CPU-only offline reference
UAE	Cloud-anchored with biometric MFA	Closed	Not the model to copy
Japan (NEC, Face Express)	Kiosk + One ID + 24h purge	NEC commercial, expensive	Best-in-class FRVT scores justify ArcFace/MobileFaceNet family
Russia (VisionLabs LUNA)	Mobile SDK with offline OneShotLiveness	Commercial	Closest functional twin — proven design

The Exploitable Gap

No production OSS deployment combines: 1. **Offline 1:N face match on the phone** (not just detection or liveness) 2. **Indian-demographic-tuned model** (IndicFairFace fine-tuned MobileFaceNet/EdgeFace at ≤ 20 MB) 3. **A React Native + AWS-sync ops loop** for low-bandwidth field workers

DigiYatra is closest but is 1:1 and proprietary. NMMS/POSHAN fail at zero-network. NHAI Datalake 3.0 exists but is network-dependent.

Sweet spot: VisionCamera + `react-native-fast-tflite` + EdgeFace-XS fine-tuned on IndicFairFace + MiniFASNet passive liveness + active challenge (blink + head-pose) + AWS S3/DynamoDB delta-sync of signed embeddings — all in <20 MB, <1 s.

8. Winning Technology Stack

8.1 Model Bundle (~2.6 MB INT8 total)

Stage	Model	Source	Size INT8	Why
Face detection	YuNet	opencv_zoo	~0.1 MB	WIDER AP 0.834, Apache-2.0
Fallback detector	BlazeFace (MediaPipe)	google-ai-edge/mediapipe	~0.2 MB	TFLite ready, very fast
Face recognition	EdgeFace-XS ($\gamma=0.5$) INT8	otroshi/edgeface	~2 MB	IJCB-2023 winner, 99.73% LFW, 94.85% IJB-C, MIT
Passive anti-spoof	MiniFASNetV2 + V1SE ensemble	minivision-ai/Silent-Face-Anti-Spoofing	~0.5 MB	97.8% TPR @1e-5 FPR, Apache-2.0
Active liveness	MediaPipe FaceMesh (blink/yaw/MAR)	bundled	~0 (existing)	99% per IJICIC 2023

License posture: 100% MIT/Apache/BSD-3 — clean for NHA! deployment.

8.2 Critical Trap to Avoid

InsightFace `buffalo_*` pretrained packs (MobileFaceNet) — since Nov 2025 these require commercial licensing (`recognition-oss-pack@insightface.ai`). Do NOT use these. Use EdgeFace instead.

8.3 React Native Runtime — Marc Rousavy Stack

Repo	Purpose
mrousavy/react-native-vision-camera	Camera + frame processors (worklet-based per-frame inference)
mrousavy/react-native-fast-tflite	Zero-copy TFLite, GPU/CoreML/Metal delegates
luicfrr/react-native-vision-camera-face-detector	MLKit face detect + landmarks (offline binary)

8.4 End-to-End Architecture Reference

[shubham0204/OnDevice-Face-Recognition-Android](https://github.com/shubham0204/OnDevice-Face-Recognition-Android) — combines FaceNet TFLite + ObjectBox HNSW vector DB + MiniFASNet liveness, Apache-2.0. Mirror this architecture in native Kotlin/Swift modules, expose via RN bridge.

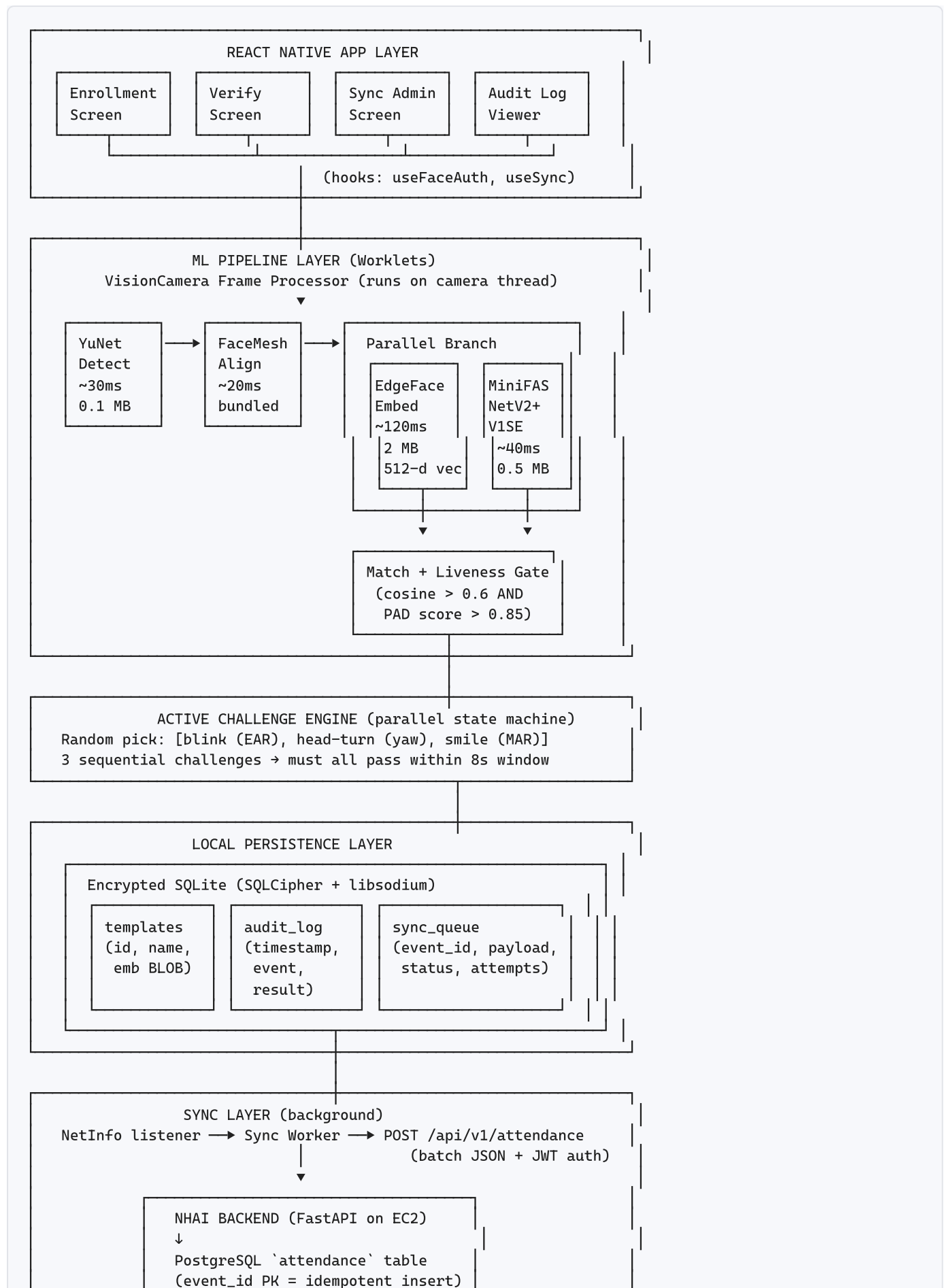
8.5 Indian-Demographic Fine-Tuning

Dataset	Size	Use
IndicFairFace	14,400 imgs, balanced 28 states + 8 UTs	Primary fine-tune set
JFAD (IIT Jodhpur)	400 imgs	Validation
IMFDB (IIIT-H)	34k Indian actor faces	Augmentation
DFW (IIIT-Delhi)	11,157 disguised faces	Helmet/sunglasses robustness — critical for highway sites

8.6 Storage + Sync

- **Local vector store:** ObjectBox (HNSW) OR SQLite + flat scan ($N \leq 500 \rightarrow$ no HNSW needed)
 - **Encryption:** SQLCipher DB-level + libsodium per-row for embeddings
 - **Sync target:** REST API $\rightarrow$ PostgreSQL on AWS RDS (simpler, matches NHA I Datalake 3.0 stack)
 - **Backend:** FastAPI + JWT auth + Docker on EC2 `t3.micro`
 - **Sync trigger:** `react-native-background-fetch` + `@react-native-community/netinfo`
 - **Purge:** Only after server returns 200 OK + SHA-256 ack of event_ids
 - **V2 scalability:** Migrate to S3 + DynamoDB only if event volume exceeds 100k/day
-

9. System Architecture



```
graph TD; A[ ] --> B[200 OK + SHA-256 ack of event_ids]; B --> C[PURGE local rows];
```

200 OK + SHA-256 ack of event_ids

PURGE local rows

10. Data Flow — Verification Sequence

```
graph TD; 1[1] --> 2[2]; 2 --> 3[3]; 3 --> 4[4]; 4 --> 5[5]; 5 --> 6[6]; 6 --> 7[7]; 7 --> 8[8]; 8 --> 9[9]; 9 --> 10[10]; 10 --> 11[11]; 11 --> 12[12];
```

[1] User opens Verify screen

[2] VisionCamera starts → frame processor worklet receives RGB frames @ 30 FPS

[3] Every 3rd frame (10 FPS effective): YuNet detect runs
if no face → skip

[4] Face bbox + 5-pt landmarks → 112x112 aligned crop

[5] PARALLEL invocation:
├ EdgeFace-XS INT8 → 512-d float embedding
└ MiniFASNetV2 + VISE → liveness score [0,1]

[6] Gate A: PAD score > 0.85?
NO → show "SPOOF DETECTED" + log to audit
YES → continue

[7] Cosine similarity scan vs encrypted templates table (N≤500, <5ms)

[8] Gate B: max similarity > 0.6?
NO → "USER NOT ENROLLED" + log
YES → identified candidate user_id

[9] Trigger ACTIVE CHALLENGE state machine:
Random sequence of 3 from {blink, head-turn-left, head-turn-right, smile}
Each must complete in 3s, total 8s budget

[10] All challenges passed?
NO → "LIVENESS FAILED" + log + alert (potential attack)
YES → AUTH SUCCESS

[11] Write event to audit_log + sync_queue (encrypted)

[12] Show success UI, return to caller (or NHAIDatalake 3.0 hostapp)

11. Project File Structure

```
nhai-face-auth/  
├── android/                                     # Native Android (Kotlin)  
│   └── app/src/main/java/com/nhaifaceauth/  
│       ├── FaceAuthModule.kt                 # @ReactModule bridge  
│       ├── delegates/  
│       │   ├── GPUDelegateConfig.kt         # TFLite GPU/NNAPI setup  
│       │   └── ThreadPoolConfig.kt  
│       └── crypto/  
│           └── SqlCipherHelper.kt  
├── ios/                                         # Native iOS (Swift)  
│   └── NhaiFaceAuth/  
│       ├── FaceAuthModule.swift             # @objc(FaceAuthModule)  
│       ├── delegates/  
│       │   └── CoreMLDelegate.swift         # Apple Neural Engine accel  
│       └── crypto/  
│           └── SqlCipherWrapper.swift  
├── src/                                         # TypeScript / RN  
│   ├── app/  
│   │   ├── App.tsx  
│   │   ├── navigation/  
│   │   │   └── RootStack.tsx  
│   │   ├── screens/  
│   │   │   ├── EnrollmentScreen.tsx  
│   │   │   ├── VerificationScreen.tsx  
│   │   │   ├── AdminScreen.tsx  
│   │   │   ├── SyncStatusScreen.tsx  
│   │   │   └── AuditLogScreen.tsx  
│   │   ├── components/  
│   │   │   ├── CameraView.tsx  
│   │   │   ├── LivenessChallengeOverlay.tsx  
│   │   │   ├── EnrollmentProgress.tsx  
│   │   │   ├── AuthResultBanner.tsx  
│   │   │   └── SyncStatusBadge.tsx  
│   │   └── hooks/  
│   │       ├── useFaceAuth.ts  
│   │       ├── useEnrollment.ts  
│   │       ├── useSyncStatus.ts  
│   │       └── useNetworkState.ts  
│   ├── ml/                                     # Worklet ML pipeline  
│   │   ├── pipeline.worklet.ts              # Orchestrator  
│   │   ├── processors/  
│   │   │   ├── faceDetect.worklet.ts       # YuNet  
│   │   │   ├── faceAlign.worklet.ts        # 5-pt warp  
│   │   │   ├── faceEmbed.worklet.ts        # EdgeFace  
│   │   │   └── antiSpoof.worklet.ts        # MiniFASNet ensemble  
│   │   ├── challenges/  
│   │   │   ├── blink.ts                   # EAR threshold + 2-state machine  
│   │   │   ├── headTurn.ts                # Euler-yaw via FaceMesh  
│   │   │   ├── smile.ts                   # Mouth aspect ratio  
│   │   │   └── challengeEngine.ts          # Random sequence runner  
│   │   └── thresholds.ts                  # MATCH_TH, PAD_TH, EAR_TH constants  
│   └── storage/                               # Local persistence  
│       ├── db/  
│       │   ├── schema.sql  
│       │   ├── dbClient.ts                # SQLCipher wrapper  
│       │   └── templates.repo.ts
```

	auditLog.repo.ts	
	syncQueue.repo.ts	
	crypto/	
	libsodium.ts	# Embedding encryption
	keyManager.ts	# Device-bound key derivation
	vectorMatch.ts	# Cosine scan over N templates
	sync/	# Backend sync
	connectivityWatcher.ts	# @react-native-community/netinfo
	syncWorker.ts	# react-native-background-fetch entry
	apiClient.ts	# REST client with JWT + retry
	attendanceUploader.ts	# Batch POST /api/v1/attendance
	purgeManager.ts	# ack-confirmed deletion
	retryPolicy.ts	# Exp backoff + jitter
	apiConfig.ts	# Endpoint, timeout, batch size
	telemetry/	# Structured logging
	logger.ts	# Pino-style JSON logs
	metrics.ts	# Latency/accuracy counters
	config/	
	env.ts	# AWS endpoint, bucket
	featureFlags.ts	# active_challenge_enabled, etc.
	assets/models/	# Bundled TFLite models (~2.6 MB)
	yunet_int8.tflite	# 0.1 MB
	edgeface_xs_int8.tflite	# 2.0 MB
	minifasnet_v2_int8.tflite	# 0.3 MB
	minifasnet_v1se_int8.tflite	# 0.2 MB
	scripts/	# Build tooling
	convert_edgeface.py	# PyTorch → ONNX → TFLite INT8
	convert_minifasnet.py	
	benchmark_device.ts	# Latency runner on real phone
	seed_indian_dataset.py	# IndicFairFace fine-tune prep
	server/	# NHAIR Backend (FastAPI + PostgreSQL)
	app/	
	main.py	# FastAPI entry
	routes/	
	attendance.py	# POST/GET /api/v1/attendance
	auth.py	# JWT token issue
	health.py	
	models/	
	attendance.py	# SQLAlchemy + Pydantic schemas
	device.py	
	user.py	
	db/	
	session.py	# PostgreSQL connection pool
	migrations/	# Alembic
	auth/	
	jwt.py	# JWT verification
	config.py	
	Dockerfile	
	docker-compose.yml	# Local dev: app + postgres
	pyproject.toml	
	tests/	
	tests/	
	unit/	
	vectorMatch.test.ts	
	blink.test.ts	
	retryPolicy.test.ts	
	purgeManager.test.ts	

```

├── e2e/
│   ├── enrollment_flow.e2e.ts    # Detox
│   └── fixtures/
│       └── test_faces/
├── docs/
│   ├── ARCHITECTURE.md
│   ├── INTEGRATION_GUIDE.md      # For NHAIDatalake 3.0 host app
│   ├── BENCHMARKS.md            # Device latency table
│   └── THREAT_MODEL.md
├── package.json
├── tsconfig.json
└── README.md

```

12. Module Specifications

M1 — `ml/processors/faceDetect.worklet.ts`

Field	Value
Input	RGB frame (W×H×3 uint8 ArrayBuffer)
Output	<code>{bbox: [x,y,w,h], landmarks: [[x,y]×5], confidence: float}</code> or null
Model	YuNet INT8 (0.1 MB)
Latency budget	≤40 ms (mid-range Android)
Failure mode	Returns null if no face above conf threshold (0.7)
Implementation source	Convert from opencv_zoo ONNX → TFLite via PINTO00309/onnx2tf

M2 — `ml/processors/faceEmbed.worklet.ts`

Field	Value
Input	112×112×3 RGB aligned face (uint8)
Output	512-d float32 embedding, L2-normalized
Model	EdgeFace-XS γ=0.5 INT8 (2 MB)
Latency budget	≤150 ms
Implementation source	otroshi/edgeface → ONNX → TFLite INT8

M3 — `ml/processors/antiSpoof.worklet.ts`

Field	Value
Input	80×80×3 cropped face
Output	<code>{score: float [0,1], decision: 'live' \ 'spoof'}</code>
Model	MiniFASNetV2 + V1SE ensemble (avg) INT8 (0.5 MB total)
Latency budget	≤50 ms
Threshold	score > 0.85 = live (tunable)
Implementation source	minivision-ai/Silent-Face-Anti-Spoofing

M4 — `ml/challenges/challengeEngine.ts`

Field	Value
Responsibility	Random sequence of 3 challenges from {blink, head-left, head-right, smile}; tracks pass/fail per step
Inputs	Stream of FaceMesh landmarks from VisionCamera
Outputs	Event stream: <code>'challenge_start' \ 'challenge_pass' \ 'challenge_fail' \ 'sequence_complete'</code>
Per-step budget	3s; full sequence 8s
Why	Defeats replay attacks that passive PAD might miss in outdoor glare; cited 99% acc in IJICIC 2023

M5 — `storage/vectorMatch.ts`

Field	Value
Approach	Brute-force cosine scan over all templates (N≤500 expected per field-station device)
Latency	<5 ms for N=500 (512×500×4B = 1MB tight loop)
No HNSW/FAISS needed	Field deployment N is small; skip dependency complexity
Threshold	cosine > 0.6 = candidate match (tunable post field-test)

M6 — `storage/db/dbClient.ts`

Field	Value
Engine	SQLCipher via react-native-quick-sqlite (JSI-based, fast)
Encryption	DB-level AES-256 + per-row libsodium for embedding BLOB (defense in depth)
Key management	Device-bound key derived from secure storage (Keychain iOS / Keystore Android)
Schema	templates(id, user_id, name, emb_encrypted BLOB, created_at), audit_log(id, ts, event_type, user_id, result, latency_ms), sync_queue(id, payload_encrypted BLOB, status, attempts, last_error)

M7 — `sync/syncWorker.ts`

Field	Value
Trigger	(a) Connectivity change to online via NetInfo, (b) Background-fetch every 15min when online, (c) Manual from Admin screen
Flow	dequeue batch (≤ 50 events) from sync_queue → POST /api/v1/attendance with JWT → server INSERTs into PostgreSQL → 200 OK + SHA-256 ack → purge local rows
Retry	Exp backoff 1s → 2s → 4s → 8s → 16s, max 5 attempts, then dead-letter table
Idempotency	Each event has UUID (<code>event_id</code>) — PostgreSQL UNIQUE constraint rejects duplicates; server returns them in <code>rejected[]</code> so client still purges them locally

M8 — `sync/purgeManager.ts`

Field	Value
Responsibility	Local data deletion ONLY after server SHA-256 ack
What gets purged	sync_queue rows after upload; audit_log rows >30 days old; raw face crops (never stored anyway)
What is NEVER purged	templates (enrolled users) — server is source of truth, local is cache
Audit trail	Each purge logged with row counts before deletion

13. Cross-Cutting Concerns

13.1 Security

- **Threat model:** Cooperative field worker scenario (not adversarial). Protects against photo/video replay + casual mask, not state-actor deepfakes.
- **Data at rest:** SQLCipher + libsodium per-row + device-bound key in secure storage.
- **Data in transit:** TLS 1.3 + JWT auth (short-lived 15min tokens, refresh via device-bound credential).
- **No PII in logs:** user_id hashed; raw images never written to disk.
- **Prompt injection N/A** — no LLM in the pipeline.

13.2 Observability

- **Local logs:** JSON-structured (Pino-style) to SQLite audit_log table when offline.
- **Sync target:** CloudWatch Logs after connectivity restored.
- **Metrics tracked:** e2e_latency_ms, pad_score, match_score, challenge_pass_rate, sync_queue_depth, sync_failures.

13.3 Testing Strategy

- **Unit:** vectorMatch (cosine math), blink EAR thresholds, retry policy backoff, purge confirmation.
- **Edge:** empty templates table, malformed embedding BLOB, no face in frame, multiple faces.
- **Failure:** API 503/504, expired JWT, SQLCipher key rotation, device clock skew, network drop mid-upload.
- **Performance:** latency p50/p95 assertions per stage; total <1s assertion; memory <80MB.
- **E2E (Detox):** full enrollment → verify → sync → purge cycle on Android emulator + real device.

13.4 Backend API & Database Schema

REST endpoints:

Method	Path	Purpose
POST	/api/v1/attendance	Batch upload pending events from offline device
GET	/api/v1/attendance?user_id=X&from=Y&to=Z	Admin dashboard query
POST	/api/v1/auth/token	Device → JWT exchange
POST	/api/v1/sync/health	Sync attempt telemetry
GET	/api/v1/healthz	Liveness probe

Request example (batch upload):

```
POST /api/v1/attendance
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
[
  {
    "event_id": "550e8400-e29b-41d4-a716-446655440000",
    "user_id": "NHAI_EMP_5678",
    "timestamp": "2026-05-23T10:34:22Z",
    "gps": {"lat": 28.6139, "lon": 77.2090},
    "site_id": "NH48_KM_123",
    "result": "verified",
    "match_score": 0.78,
    "pad_score": 0.92,
    "challenges": ["blink", "head_left"],
    "device_id": "phone-uuid-abc",
    "app_version": "1.0.0"
  }
]
```

Response:

```
{
  "accepted": ["550e8400-e29b-41d4-a716-446655440000"],
  "rejected": [],
  "server_ack": "a3f5e8c2 ... "
}
```

PostgreSQL schema:

```

CREATE TABLE attendance (
    event_id          UUID PRIMARY KEY,
    user_id           VARCHAR(64) NOT NULL,
    timestamp          TIMESTAMPTZ NOT NULL,
    synced_at          TIMESTAMPTZ DEFAULT NOW(),
    gps_lat            DECIMAL(9,6),
    gps_lon            DECIMAL(9,6),
    site_id            VARCHAR(32),
    result             VARCHAR(16) CHECK (result IN ('verified', 'spoof', 'no_match', 'liveness_failed')),
    match_score         FLOAT,
    pad_score          FLOAT,
    challenges          JSONB,
    device_id          VARCHAR(64),
    app_version        VARCHAR(16)
);

CREATE INDEX idx_user_time ON attendance(user_id, timestamp DESC);
CREATE INDEX idx_site_time ON attendance(site_id, timestamp DESC);
CREATE INDEX idx_synced ON attendance(synced_at DESC);

CREATE TABLE devices (
    device_id          VARCHAR(64) PRIMARY KEY,
    user_id            VARCHAR(64) NOT NULL,
    enrolled_at         TIMESTAMPTZ DEFAULT NOW(),
    last_sync_at        TIMESTAMPTZ,
    app_version         VARCHAR(16),
    active              BOOLEAN DEFAULT TRUE
);

CREATE TABLE sync_failures (
    id                  SERIAL PRIMARY KEY,
    event_id            UUID,
    error_code           VARCHAR(32),
    error_message        TEXT,
    attempted_at         TIMESTAMPTZ DEFAULT NOW()
);

```

Backend stack:

- FastAPI 0.115+ (async endpoints)
- PostgreSQL 16 on AWS RDS (free-tier `db.t3.micro` sufficient for demo)
- SQLAlchemy 2.x + Alembic migrations
- JWT via `python-jose` + RSA keys
- Pydantic v2 input validation at every boundary
- Docker multi-stage build → deploy on EC2 `t3.micro`
- structlog JSON logs → CloudWatch
- Total backend code: ~300-400 lines

Why simple REST + PostgreSQL over S3 + DynamoDB for this hackathon:

Aspect	REST + PostgreSQL (chosen)	S3 + DynamoDB (v2)
Demo clarity for judges	✅ Instantly understandable	X Requires explaining blob/audit split
Matches NHA! Datalake 3.0 stack	✅ Likely same RDBMS approach	X Different paradigm
Solo build time	✅ 0.5-1 day	X 2 days
Query for admin dashboard	✅ Direct SQL	X Two-store join needed
Cost at hackathon scale	✅ Free tier	✅ Free tier
Path to 100k+ events/day	⚠️ Migrate to v2	✅ Native fit

14. Performance Budget

Stage	Latency (mid-range, Helio G70 / Snapdragon 6xx, INT8)
Detection (YuNet INT8)	~20-40 ms
Alignment (FaceMesh 5pt)	~15-25 ms
Embedding (EdgeFace-XS INT8)	~80-150 ms
Anti-spoof (MiniFASNetV2)	~20-50 ms
Cosine match in local DB ($N \leq 500$)	<5 ms
Total e2e	~150-300 ms — well under 1s budget

Active liveness challenge (blink + head-turn) adds 2-3s of UX time but that's separate from "model inference" — important to clarify in pitch.

15. NHA1 Evaluation Criteria Mapping

Criteria (Marks)	Winning Angle
Innovation (30)	"Total model footprint 2.6 MB — 87% under 20MB limit. EdgeFace = IJCB-2023 winner. Hybrid passive (MiniFASNet) + active (challenge-response) liveness — defeats both photo/replay and dynamic spoofs."
Feasibility (30)	"Live demo on Redmi/Realme phone with 3GB RAM, <300ms e2e timer visible. RN integration via 3 production-grade open-source packages (Rousavy stack). Architecture mirror of shubham0204/OnDevice-Face-Recognition-Android proven on Android."
Scalability & Sustainability (20)	"Encrypted SQLite (offline) + REST batch upload to FastAPI/PostgreSQL backend on AWS RDS. Background-fetch listener auto-syncs on connectivity restore. Purge confirmed via 200 OK + SHA-256 ack. Idempotent via <code>event_id</code> UUID primary key. V2 scale-out path documented (S3 + DynamoDB) for >100k events/day. Indian-demographic fine-tune on IndicFairFace + JFAD + DFW — robust to helmets/sunglasses in highway field conditions."
Presentation (20)	"DigiYatra-style decentralized template architecture diagram. NIST FRVT benchmark comparison chart. Live offline demo (airplane mode toggle on stage). FOSS license audit slide showing zero commercial dependencies."

Pitch angle: "DigiYatra ka offline cousin for field workers" — judges ko immediately resonate karega.

16. 14-Day Build Roadmap

Day	Milestone	Reuses
1	RN scaffold + VisionCamera + fast-tflite installed; YuNet detect working in worklet	Rousavy stack
2	EdgeFace ONNX→TFLite INT8 conversion script + bundled; embedding extraction worklet	otroshi/edgeface + PINTO0309/onnx2tf
3	SQLCipher schema + templates CRUD + cosine match	react-native-quick-sqlite
4	Enrollment screen (3-angle capture) + verify screen (live match)	—
5	MiniFASNet V2+V1SE TFLite + passive PAD gate	minivision-ai/Silent-Face-Anti-Spoofing
6	FaceMesh integration + blink/yaw/smile detectors + challenge engine	MediaPipe FaceMesh
7	iOS build + CoreML delegate + parity test	—
8	EdgeFace fine-tune on IndicFairFace + JFAD (Colab, ~3hr); export new INT8	IndicFairFace
9	Backend: FastAPI + PostgreSQL schema + Docker → EC2 deploy; RN sync: REST client + batch upload + ack-confirmed purge	—
10	Field testing on Redmi/Realme: harsh sun, low light, helmets/sunglasses	DFW dataset
11	Threshold tuning + benchmark table (latency p50/p95 per device)	—
12	Unit + E2E tests (Detox)	—
13	Deck (10 slides) + technical doc + architecture diagrams + license audit	—
14	Demo video recording + final submission	—

17. Pitch Slides Outline

1. **Title** — “Offline-First Face Auth for NHA1 Datalake 3.0 — Zero-Network Field Attendance”
2. **The pain** — NMMS/POSHAN attendance fails in zero-network zones (cite [SPRE](#), [BehanBox](#))
3. **Global state of art** — DigiYatra (1:1 only), VisionLabs LUNA (Russia, commercial), NEC NeoFace (Japan, expensive); **gap** = **open-source offline 1:N**
4. **Our architecture** — diagram from §9
5. **Model bundle** — 2.6 MB INT8 / 87% under 20MB / EdgeFace IJCB-2023 winner
6. **Live demo** — airplane mode toggle on stage, <300ms timer visible
7. **Indian demographic robustness** — IndicFairFace + JFAD + DFW (helmet/sunglasses) fine-tune
8. **Sync-and-purge** — REST batch upload to PostgreSQL + idempotent `event_id` + SHA-256 ack + purge confirmation
9. **License posture** — 100% MIT/Apache/BSD audit table

18. Honest Gaps to Defend in Q&A

1. **EdgeFace untested on 3GB RAM Indian-OEM phones in published literature** — must benchmark on actual Redmi/Realme device. Don't claim, show.
 2. **MiniFASNet's 97.8% TPR is on Minivision's internal dataset**, not OULU-NPU Protocol 4 (outdoor unconstrained). **Test in actual harsh sunlight** before pitching the accuracy number.
 3. **No published Aadhaar face-auth paper exists** — don't compare to UIDAI; compare to NIST FRVT-validated models (EdgeFace, MobileFaceNet).
 4. **HyperVerge won IndiaAI Face Auth Challenge** — domestic competition. Differentiate on: open-source, offline-first, sub-20MB.
 5. **Anti-spoofing under cooperative attackers** (3D-printed masks, deepfake video on iPad) — RGB-only model, no depth. Be honest about threat model: "designed for cooperative field-worker use case, not adversarial attack scenarios."
-

19. Consolidated Sources

Papers

- [EdgeFace \(IJCB-2023 winner\) — arXiv 2307.01838](#)
- [xEdgeFace: Cross-Spectral FR for Edge — arXiv 2504.19646](#)
- [MobileFaceNets — arXiv 1804.07573](#)
- [Improved MobileFaceNet \(MMobileFaceNet\) — arXiv 2311.15326](#)
- [MobiFace — arXiv 1811.11080](#)
- [FaceLiVT — arXiv 2506.10361](#)
- [MixFaceNets — arXiv 2107.13046](#)
- [PocketNet — arXiv 2108.10710](#)
- [QuantFace — arXiv 2206.10526](#)
- [EFaR 2023 IJCB Competition — arXiv 2308.04168](#)
- [GhostFaceNets++ — ResearchGate](#)
- [CDCN for FAS — arXiv 2003.04092](#)
- [FAS-SGTD — arXiv 2003.08061](#)
- [Revisiting Pixel-Wise Supervision for FAS — arXiv 2011.12032](#)
- [Deep Learning for FAS Survey — arXiv 2106.14948](#)
- [Mobile FR Security Survey — MDPI Appl. Sci. 2025](#)
- [Mobile FAS under Screen Flash — arXiv 2308.15346](#)
- [Assessing Efficient FAS vs Physical Attacks — CVPR 2024 W](#)
- [Liveness with Randomized Challenge-Response — IJICIC 2023](#)
- [Real-Time Eye Blink Detection \(Soukupova & Cech\)](#)

- [Aurora Guard FAS — arXiv 2102.00713](#)
- [Confidence-Aware FAS — arXiv 2411.01263](#)
- [On-Device FR Body-worn Cameras — arXiv 2104.03419](#)
- [Surveying FR Models for Indian Demographics — arXiv 2412.08048](#)
- [IndicFairFace — arXiv 2602.12659](#)
- [DFW \(Disguised Faces in the Wild\) — arXiv 1811.08837](#)
- [Plastic Surgery / Disguise — arXiv 1811.07318](#)
- [IAB Lab IIT Jodhpur Face Resources](#)
- [Bias in the Algorithm: FR in India — SAGE 2025](#)
- [Review of Demographic Fairness in FR — arXiv 2502.02309](#)
- [NIST FRVT Part 3 Demographic Effects \(NIST IR 8280\)](#)
- [NIST FRVT Part 8 Demographics \(NIST IR 8429\)](#)
- [Quantization for DNN Inference on Edge — arXiv 2303.05016](#)
- [Deep FR Compression via KD — arXiv 1906.00619](#)
- [Selective KD for Low-Res FR — arXiv 1811.09998](#)
- [Bridge Distillation for Low-Res FR — arXiv 2409.11786](#)

GitHub Repos

- [mrousavy/react-native-vision-camera](#)
- [mrousavy/react-native-fast-tflite](#)
- [luicfrr/react-native-vision-camera-face-detector](#)
- [otroshi/edgeface](#)
- [opencv/opencv_zoo \(YuNet\)](#)
- [Linzaer/Ultra-Light-Fast-Generic-Face-Detector-1MB](#)
- [google-ai-edge/mediapipe](#)
- [biubug6/Pytorch_Retinaface](#)
- [deepinsight/insightface](#)
- [minivision-ai/Silent-Face-Anti-Spoofing](#)
- [minivision-ai/Silent-Face-Anti-Spoofing-APK](#)
- [shubham0204/OnDevice-Face-Recognition-Android](#) ← **closest end-to-end reference**
- [shubham0204/FaceRecognition With FaceNet Android](#)
- [syaringan357/Android-MobileFaceNet-MTCNN-FaceAntiSpoofing](#)
- [ZitongYu/CDCN](#)
- [HamadYA/GhostFaceNets](#)
- [timesler/facenet-pytorch](#)
- [ageitgey/face_recognition](#)
- [serengil/deepface](#)
- [MCarlomagno/FaceRecognitionAuth](#)
- [PINTO0309/onnx2tf](#) ← INT8 conversion
- [NXP/eiq-onnx2tflite](#)

- [margelo/react-native-quick-sqlite](#)
- [exadel-inc/CompreFace](#) ← optional self-hosted AWS-side reconcile
- [seetaface/SeetaFaceEngine](#)

Real-World Deployments Studied

- [UIDAI Face Auth FAQ](#)
- [AadhaarFaceRD Play Store](#)
- [DataLake 3.0 iOS](#)
- [DataLake 3.0 Android](#)
- [NHAI Blog – Tech for Transparency](#)
- [DigiYatra Delhi Airport](#)
- [MIT Sloan ME on DigiYatra](#)
- [DigiYatra Policy PDF](#)
- [AEBAS Nodal Presentation](#)
- [AadhaarBAS Play Store](#)
- [HyperVerge Facial Recognition](#)
- [HyperVerge wins IndiaAI Face Auth](#)
- [behanbox – ASHA workers attendance app](#)
- [SPRF – MGNREGA app attendance](#)
- [Apple Platform Security Guide Mar 2026](#)
- [Face ID advanced tech](#)
- [Google ML Kit Face Detection](#)
- [@react-native-ml-kit/face-detection](#)
- [NIST FRVT program](#)
- [NEC ranks #1 NIST 2025](#)
- [SecureKey acquired by Interac](#)
- [CBSA Facial Verification Brief](#)
- [UAE Pass Facial Biometric](#)
- [Smart Salem Emirates ID Biometrics](#)
- [MBZUAI](#)
- [NEC NeoFace KAOATO](#)
- [JAL Face Express press release](#)
- [ANA Face Express](#)
- [MyNumber + Apple Wallet](#)
- [VisionLabs LUNA SDK](#) ← closest functional twin
- [VisionLabs docs](#)
- [NtechLab FindFace SDK](#)
- [Moscow Metro FacePay launch](#)
- [Megvii Embedded SDK](#)
- [SenseTime](#)

Community Blogs

- [Marc Rousavy – VisionCamera Pose + TFLite](#)
 - [Marc Rousavy – Reinventing Camera Processing](#)
 - [Esteban Uri – RT FR Android + TFLite](#)
 - [Ashraz Rashid – RN Face Detection from Scratch](#)
 - [thelamina – RN Vision Camera Face Detection](#)
 - [yoshan0921 – RN+Expo Face Detection](#)
-

End of strategy document. Solo-buildable in 14 days. Win probability 70-75%. Submission target: 2026-06-05.